

Car Repair NFT — README

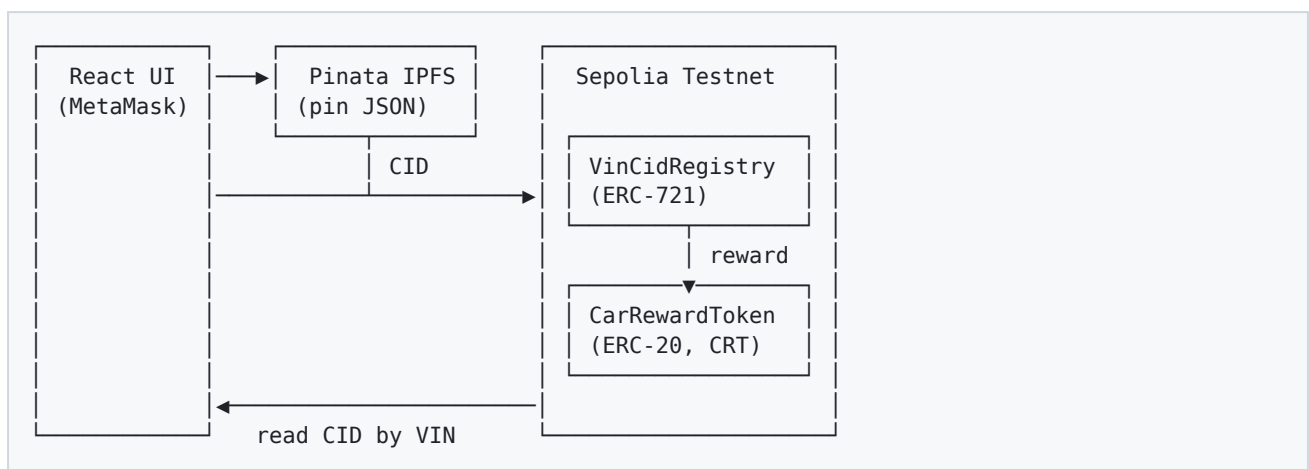
Generated from README.md

Car Repair NFT

A dApp that records vehicle repair history on-chain. Each car — identified by its 17-character VIN — gets a single NFT whose tokenURI points at the latest IPFS CID for that car's repair metadata. Only a designated minter address (a registry operator, separate from the contract owner) may register new VINs; the NFT is assigned to a recipient argument (typically the car owner's wallet). Updating a record rewrites the NFT's URI in place (no new mint), and the registry pays out an ERC-20 reward (CRT) to the recipient on the initial mint.

Flow diagram: <https://excalidraw.com/#json=zV5wVQt8GJoK-GYiO-DQn,5mQBcqQrwVfjEm3sxA0Dyw>

Architecture



Write flow:

1. The **minter** operator (e.g. a registry admin) connects MetaMask (Sepolia) in the frontend. To register a new VIN, the connected wallet must equal the contract's minter().
2. On submit, the frontend pins a JSON metadata object to IPFS via Pinata and gets back a CID.
3. The frontend calls `VinCidRegistry.storeCid(vin, cid, recipient)`. The first call for a VIN mints its NFT to recipient (the car owner). Later calls update the tokenURI — updates are open in this POC build (anyone can call them).
4. On mint only, the registry attempts to transfer `rewardAmount` of CRT to the recipient (best-effort — silent on failure, so the write still succeeds if funding is empty).
5. To read history, the UI calls `getCidByVin(vin)` and fetches the JSON from `https://gateway.pinata.cloud/ipfs/<cid>`.

Repository layout

```

car_nft/
├── contracts/
│   ├── car_nft_sc.sol          # VinCidRegistry (ERC-721 URI storage)
│   └── car_reward_token.sol    # CarRewardToken (ERC-20, "CRT")
├── frontend/
│   ├── public/
│   │   ├── index.html
│   │   └── _redirects          # SPA fallback (Netlify-style)
│   ├── src/
│   │   ├── App.js             # Search + create/update forms
│   │   ├── components/
│   │   │   └── MetaMaskLogin.jsx
│   │   └── utils/
│   │       ├── contract_abi.json
│   │       ├── contract_utils.js      # chainId → contract address
│   │       ├── pinata_ipfs_nft_service.js # IPFS pin + on-chain write
│   │       └── validation.js          # VIN / CID / car-data checks
│   ├── package.json
│   └── vercel.json             # SPA rewrite for Vercel

```

Development environment

The repo includes a [VS Code Dev Container](#) (also usable as a GitHub Codespace). Opening the repo in it provisions:

- Base image `mcr.microsoft.com/devcontainers/base:ubuntu-24.04`, Node 22, zsh (as the default shell), and the GitHub CLI.
- VS Code extensions: ESLint, Solidity (JuanBlanco.solidity), Prettier (with format-on-save enabled).
- Port 3000 auto-forwarded and opened in a browser for the CRA dev server.

`postCreateCommand` only runs `npm install` inside `frontend/`. It does **not** install the root-level dependencies (Hardhat, OpenZeppelin, etc.), so after the container finishes building, run `npm install` once at the repo root before using any of the `npm run compile` / `npm run node` / `npm run deploy:*` scripts below.

Smart contracts

`VinCidRegistry` (contracts/car_nft_sc.sol)

ERC-721 with URI storage. Token id is `uint256(keccak256(bytes(vin)))`, so each VIN maps to exactly one NFT.

Two roles:

- `owner()` — admin (set at deploy via `Ownable`). Configures reward params and the minter.
- `minter` — registry operator authorized to register new VINs. Set at deploy and changeable by owner via `setMinter(address)`. Updates to existing records are open in this POC and do not require either role.

Key functions:

- `storeCid(string vin, string cid, address recipient)` — mint or update. Requires `vin` length 17 and non-empty `cid`. On a new mint, `msg.sender` must be `minter` and `recipient` must be non-zero; the NFT is minted to `recipient` and the CRT reward is paid to `recipient`. On updates, `recipient` is ignored. Emits `CidStored(vin, cid, tokenId)`.
- `minter() → address` / `setMinter(address)` — read or update the minter (`setMinter` is owner-only). Emits `MinterChanged(previousMinter, newMinter)`.
- `getCidByVin(string vin) → string` — latest CID for a VIN.
- `getAllVins() → string[]` / `getAllCidsAsList() → string[]` — enumerate the registry.
- `setRewardToken(address)` / `setRewardAmount(uint256)` — owner-only configuration.

- `withdrawToken(IERC20, address, uint256)` — owner-only; recover ERC-20 funds held by the registry.

`CarRewardToken` (contracts/car_reward_token.sol)

Standard OpenZeppelin ERC-20 named CarRewardToken (symbol CRT). Mints 1,000,000,000 CRT to the deployer at construction; the owner can `mint(to, amount)` more later.

Deploying the smart contracts

The repo ships a Hardhat setup at the root. Two flavors:

- **Local** — a persistent EVM node on `127.0.0.1:8545` with 20 prefunded test accounts. Fast, free, no faucet needed. Use this for development on the dev branch.
- **Sepolia** — Ethereum's public testnet. Requires a Sepolia RPC endpoint, a funded deployer wallet, and (optionally) an Etherscan API key for source verification. Used for production via `main` → Vercel.

Both flavors run the same `scripts/deploy.js`: deploy CarRewardToken, then VinCidRegistry, then fund the registry with CRT and set the per-mint reward.

The root project runs on **Hardhat 3** as an ESM package (`"type": "module"` in `package.json`) — `hardhat.config.js` and `scripts/deploy.js` use `import/export`, not `require/module.exports`. The plugin bundle is `@nomicfoundation/hardhat-toolbox-mocha-ethers` (ethers + Mocha stack). `npm run compile` also generates a `types/` directory (TypeChain bindings); like `artifacts//cache/`, it's regenerated on every compile and gitignored.

Prerequisites

One-time install at the repo root:

```
npm install
```

Scripts available from the repo root (`package.json`):

Command	What it does
<code>npm run compile</code>	Compile contracts (output → <code>artifacts/</code>)
<code>npm run node</code>	Start a persistent local EVM JSON-RPC at <code>127.0.0.1:8545</code> (Cancun EVM, <code>chainId 31337</code>)
<code>npm run deploy:local</code>	Deploy to the running local node and auto-write the registry address into <code>frontend/.env.local</code>
<code>npm run deploy:sepolia</code>	Deploy to Sepolia using credentials from root <code>.env</code>

Configure environment

Copy [.env.example](#) to `.env` at the repo root. Leave it empty for local deploys; fill in for Sepolia / verify:

Variable	Required for	Notes
<code>SEPOLIA_RPC_URL</code>	Sepolia	Alchemy / Infura / QuickNode endpoint
<code>DEPLOYER_PRIVATE_KEY</code>	Sepolia	Dedicated test wallet — never reuse a mainnet key

ETHERSCAN_API_KEY	npx hardhat verify	Free at https://etherscan.io/myapikey
INITIAL_MINTER	optional	Defaults to deployer address
REWARD_AMOUNT	optional (default 10)	CRT per mint
REWARD_FUND	optional (default 1000000)	CRT pool funded into the registry on deploy

.env is gitignored. Never commit real values.

Option 1 — Local deploy (Hardhat, no Sepolia)

For day-to-day development. Two terminals from the repo root:

```
# Terminal 1 – leave running
npm run node

# Terminal 2 – deploys CRT + registry, funds it
npm run deploy:local
```

What `deploy:local` does:

1. Deploys CarRewardToken and VinCidRegistry.
2. Transfers REWARD_FUND CRT to the registry and sets rewardAmount.
3. Writes a deployment artifact to `deployments/localhost.json` (gitignored, ephemeral).
4. **Auto-populates** `REACT_APP_SMART_CONTRACT_ADDRESS_LOCAL` in [frontend/.env.local](#) — no copy-paste needed.

To use it from the frontend:

- In MetaMask, add the localhost network: RPC `http://127.0.0.1:8545`, chainId 31337, currency ETH.
- Import one of the test private keys printed by `npm run node` (account 0 = `0xf39Fd6e5...F92266`, the well-known Hardhat test account).
- Restart `npm start` in `frontend/` so CRA picks up the new `.env.local` value.

Dev Container / Codespaces users: MetaMask runs in your browser on the host machine, not inside the container, so it can only reach `127.0.0.1:8545` if that port is forwarded out of the container — already configured in [.devcontainer/devcontainer.json](#). If the container was already running before this was added, reload the window ("Dev Containers: Reload Window") so the new forwarded port takes effect.

ETH balance vs. NFT tokens: MetaMask shows the native ETH balance for any imported account automatically — Hardhat's test accounts start pre-funded with 10,000 ETH, so that part "just works". It does **not** do the same for the VIN NFTs minted by VinCidRegistry: MetaMask's automatic token detection only queries its backing API for a handful of well-known networks, and Hardhat's local chain isn't one of them (Sepolia isn't reliably covered either). So after a successful mint, the NFT won't just appear in MetaMask's NFTs tab. Two ways to actually see it:

- **Easiest** — click **Show All Registered NFTs** in the app. It reads directly from the contract (`getAllVins / getAllCidsAsList`), bypassing MetaMask entirely.
- **Manual MetaMask import** — NFTs tab → Import NFT, using the registry contract address (`REACT_APP_SMART_CONTRACT_ADDRESS_LOCAL`, or from `deployments/localhost.json`) and the token ID. Token IDs here aren't sequential (1, 2, 3...) — [contracts/car_nft_sc.sol](#) derives them as `uint256(keccak256(vin))`, so you need to compute the hash rather than guess it.

State persists for as long as `npm run node` keeps running. Killing the node wipes everything; the next `npm run deploy:local` produces fresh addresses.

Warning: Hardhat's test mnemonic is public knowledge — anyone running `npx hardhat node` gets the same 20 keys. Never use these accounts on any real network.

Option 2 — Sepolia deploy

Fill in `.env` first (at least `SEPOLIA_RPC_URL` and `DEPLOYER_PRIVATE_KEY`). The deployer wallet needs ~0.05 Sepolia ETH from a faucet:

- <https://www.alchemy.com/faucets/ethereum-sepolia>
- <https://sepoliafaucet.com>

Then:

```
npm run deploy:sepolia
```

What `deploy:sepolia` does:

1. Deploys CRT + registry, funds it (same as local).
2. Writes `deployments/sepolia.json` — **committed** as the team's source of truth for what's currently live.
3. Prints the registry address to paste into Vercel's UI (see [Deploying the frontend to Vercel](#) below).

Verifying source on Etherscan (recommended)

Verification unlocks **Read Contract** / **Write Contract** tabs on Etherscan — letting anyone read state and admins run owner-only functions directly from the explorer.

```
# CarRewardToken – no constructor args
npx hardhat verify --network sepolia <crt-address>

# VinCidRegistry – constructor: (rewardTokenAddress, initialMinter)
npx hardhat verify --network sepolia <registry-address> <crt-address> <initial-minter-address>
```

Re-verifying an already-verified contract is a no-op. Requires `ETHERSCAN_API_KEY` in `.env`.

Roles

Role	Set by	Authority
owner()	Ownable constructor (= deployer)	Configures <code>rewardToken</code> , <code>rewardAmount</code> , <code>minter</code> . Can <code>withdrawToken</code> .
minter	<code>INITIAL_MINTER</code> env var, or deployer if unset (changeable by owner via <code>setMinter</code>)	Authorized to register new VINs via <code>storeCid</code> (the mint path).
Anyone	—	Can update an existing VIN's CID (POC behavior; lock down later if needed).

Reference deployment (Sepolia)

The latest live addresses are always in [deployments/sepolia.json](#). Inspect on the explorer:

- [CarRewardToken on Sepolia Etherscan](#)
- [VinCidRegistry on Sepolia Etherscan](#)

After a redeploy, commit the updated `deployments/sepolia.json` and update `REACT_APP_SMART_CONTRACT_ADDRESS` in Vercel's UI (see Vercel section).

Frontend

Stack: React 18 (CRA), Material UI 5, ethers v5, web3 v4, MetaMask provider.

ethers is listed in `frontend/package.json` but is currently **unused** — all contract reads/writes go through web3 (see [frontend/src/utlis/pinata_ipfs_nft_service.js](#)). No file under `frontend/src/` imports ethers.

Environment variables

Copy [frontend/.env.example](#) to frontend/.env.local and fill it in. CRA reads REACT_APP_* from .env.local at build time and inlines them into the bundle, so **do not put truly secret keys here** — anything REACT_APP_* is visible in the browser bundle.

```
REACT_APP_PINATA_API_URL=https://api.pinata.cloud/pinning
REACT_APP_PINATA_JWT=<your-pinata-jwt>
REACT_APP_SMART_CONTRACT_ADDRESS=<sepolia-registry-address>      # leave empty if not testing against Sepolia from npm start
REACT_APP_SMART_CONTRACT_ADDRESS_LOCAL=<hardhat-registry-address> # auto-written by `npm run deploy:local`
```

The chain → contract mapping lives in [frontend/src/utlis/contract_utlis.js](#): Sepolia (0xaa36a7) and Hardhat localhost (0x7a69). To support another network, add an entry there.

For production builds on Vercel, **don't ship a .env.local** — set the same variables as Project Environment Variables in Vercel's UI instead. See [Deploying the frontend to Vercel](#).

Run locally

```
cd frontend
npm install
npm start          # http://localhost:3000
```

Other scripts: npm run build, npm test, npm run clean (nuke build + node_modules), npm run fix-install (cache-clean + reinstall).

Using the app

1. Open the app and click **Connect Wallet**. Approve in MetaMask and switch to Sepolia if prompted. The "Create or Update" panel shows the contract's current minter address and whether your connected wallet matches.
2. **Search**: enter a 17-char VIN and click Load Car NFT. The app reads the CID from the contract and fetches the metadata from Pinata's gateway. After this call the app knows whether the VIN already exists on-chain (controls which fields/buttons appear next).
3. **Register a new car (mint)**: the connected wallet must be the **minter**. Fill in VIN, **Car Owner Wallet (recipient)**, brand, model, year, issue, repair shop, mileage, then Register New Car NFT. The app pins the JSON to IPFS and calls storeCid(vin, cid, recipient). The NFT lands in the recipient's wallet; the CRT reward is also paid to the recipient.
4. **Update an existing car**: any connected wallet works (POC behavior). The Recipient field is hidden because the NFT already exists; the update is applied to that VIN's existing NFT. Click Submit Repair Update.
5. After confirmation, the tx hash links to Sepolia Etherscan.

Deploying the frontend to Vercel

The repo contains [frontend/vercel.json](#):

```
{
  "git": { "deploymentEnabled": { "dev": false } },
  "rewrites": [ { "source": "/(.*)", "destination": "/" } ]
}
```

Two things wired here:

1. **Only main deploys to Vercel**. The dev branch is for local development against a Hardhat node and never goes to Vercel — preview deploys for dev would be useless because Vercel can't reach localhost:8545.
2. **SPA rewrite** — every path falls back to / so deep-links hit index.html and React handles routing.

One-time setup (Vercel Dashboard)

1. Push the repo to GitHub.

2. <https://vercel.com/new> → import the repo.
3. **Root Directory**: frontend (the React project lives in a subfolder).
4. **Framework Preset**: Create React App (auto-detected).
5. **Build / Output**: defaults are fine (npm run build → build/).
6. **Environment Variables** — add to the **Production** scope only:

Variable	Value
REACT_APP_SMART_CONTRACT_ADDRESS	Sepolia registry address (from deployments/sepolia.json)
REACT_APP_PINATA_API_URL	https://api.pinata.cloud/pinning
REACT_APP_PINATA_JWT	Your Pinata JWT

Do **not** set REACT_APP_SMART_CONTRACT_ADDRESS_LOCAL — production users have no Hardhat node.

7. Click **Deploy**.

Subsequent pushes to main redeploy automatically. Pushes to dev (or any other branch) do nothing in Vercel.

Re-deploying after a Sepolia redeploy

When the registry address changes (every npm run deploy:sepolia):

1. Update Vercel → Project Settings → Environment Variables → REACT_APP_SMART_CONTRACT_ADDRESS (Production scope).
2. Trigger a fresh build: push any commit to main, or **Deployments** → ... → **Redeploy**. CRA only injects env vars at build time — without a fresh build the bundle keeps the old address.
3. Commit the updated deployments/sepolia.json so the team has the new addresses too.

Vercel CLI alternative

```
npm i -g vercel
cd frontend
vercel login
vercel # first run: link project, pick "frontend" as root
vercel env add REACT_APP_SMART_CONTRACT_ADDRESS production
vercel env add REACT_APP_PINATA_JWT production
vercel env add REACT_APP_PINATA_API_URL production
vercel --prod
```

A note on `public/\_redirects`

That file is the Netlify equivalent of the rewrite. It's harmless on Vercel — the vercel.json rewrites take precedence.

Troubleshooting

- **Blank page / 404 on refresh** — confirm the SPA rewrite in vercel.json is active.
- **Contract address: null** — REACT_APP_SMART_CONTRACT_ADDRESS isn't set for the target environment, or the user is on a chain other than Sepolia.
- **Pinata 401/403** — invalid, expired, or under-scoped Pinata JWT. The error surfaces the response body — check it for the exact reason.
- **Reward not received** — registry's CRT balance is empty, or rewardAmount is 0. The write itself still succeeded.
- **Secrets in the bundle** — anything prefixed REACT_APP_ ships to the browser. Scope the Pinata JWT to pinning only, and rotate if leaked.

- No contracts to compile — this is Hardhat 3's normal "already up to date" message, not an error: it means every .sol file under contracts/ already has a valid cached build, not that contracts weren't found. To force a full rebuild anyway (e.g. after a compiler/plugin change), run `npx hardhat clean && npm run compile`.
- **NFT doesn't show up in MetaMask after minting** — expected. MetaMask's NFT auto-detection doesn't cover Hardhat's local chain (or reliably Sepolia). Use the app's **Show All Registered NFTs** button to verify the mint, or manually import the NFT in MetaMask with the contract address and token ID (see [Option 1 — Local deploy](#)).

License

Apache 2.0 — see [LICENSE](#).